



Πολυτεχνείο Κρήτης - Σχολή ΗΜΜΥ
Ειδικά Θέματα Βάσεων Δεδομένων
Πρότζεκτ (Επιλογή 1)
Προθεσμία: Πέμπτη 8 Ιανουαρίου 2026
Νικόλαος Μαλαμάς (2020030180)

Εισαγωγή

Για το πρότζεκτ του μαθήματος υλοποιήθηκε η Επιλογή 1, από τις διαθέσιμες 3. Η άσκηση ζητούσε την υλοποίηση ενός λειτουργικού pipeline χρήσι του Streaming API που περιέχεται στο Apache Flink. Οι αλγόριθμοι που υλοποιήθηκαν αποτελούν την streaming εκδοχή των αλγορίθμων που παρουσιάζονται στη δημοσίευση των Liang Chen, Kai Hwang και Jian Wu ‘*MapReduce Skyline Query Processing with A New Angular Partitioning Approach*’ που αφορά στον υπολογισμό Skylines, πάνω σε Datasets (Batch streaming). Οι αλγόριθμοι βασίζονται στη μέθοδο MapReduce και αξιοποιούνται στη δημοσίευση για την εύρεση των ‘ποιοτικότερων’ κόμβων εξυπηρέτησης για διαφορετικά Web Services.

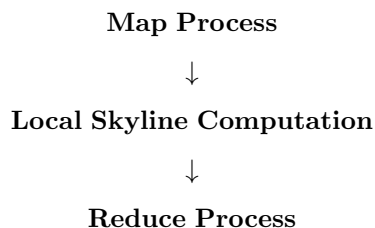
Στη δική μας υλοποίηση, στήνεται ένα σύστημα παραγωγής ερωτημάτων και πλειάδων (tuples) τα οποία κατευθύνονται σε διαφορετικά Kafka topics και ‘ρέουν’ προς το πραγματικό κομμάτι του pipeline μας για επεξεργασία (εύρεση skyline) από κάποιον από τους εξής αλγορίθμους:

- MR-Dim
- MR-Grid
- MR-Angle

Τα αποτελέσματα των υπολογισμών καταλήγουν σε ένα νέο Kafka topic και εκτυπώνονται διάφορα metrics που θα μελετήσουμε στη συνέχεια.

Υλοποίηση

Οι τρεις αλγόριθμοι που αξιοποιήθηκαν ακολουθούν τα ίδια 3 στάδια για την εκπόνηση του Global Skyline στη δημοσίευση:



και συνεπώς διαφέρουν ελάχιστα κατά την προγραμματιστική τους υλοποίηση.

Συγκεκριμένα, το μόνο σημείο που διαφέρουν οι 3 αλγόριθμοι είναι στο MapProcess και αυτό γιατί καθένας εξ αυτών χωρίζει με διαφορετικό το Data Space που ανήκουν τα tuples (datapoints). Δηλαδή, το Map κομμάτι

κάθε αλγορίθμου αξιοποιεί διαφορετική γεωμετρία για να κατανήμει τα σημεία σε partitions, ή ισοδύναμα κάνει διαφορετικό *Load Balancing*.

Το παραπάνω έχει σαν αποτέλεσμα όχι μόνο να διαφοροποιούνται οι χρόνοι μεταξύ των Map διαδικασιών των αλγορίθμων, αλλά και οι χρόνοι των αντίστοιχων Reduce διαδικασιών, για λόγους που θα μελετηθούν αναλυτικότερα στη συνέχεια.

Όπως αναφέρθηκε εν συντομία στην Εισαγωγή, δημιουργήθηκαν 2 Kafka topics, τα οποία είναι υπεύθυνα για την 'προμήθευση' του streaming συστήματος με tuples και queries. Αυτά τα topics έχουν με τη σειρά τους προμηθευτεί με τα περιεχόμενά τους από δύο ξεχωριστές από το υπόλοιπο pipeline κλάσεις. Διαφοροποιώντας τη γεννήτρια από το υπόλοιπο σύστημα μπορεί να απαιτείται η ξεχωριστή εκτέλεσή τους των δύο, ωστόσο αυτό βοηθάει στο διαχωρισμό της λογικής του συστήματος και της γέννησης δεδομένων και ερωτημάτων. Κατά αυτόν τον τρόπο μπορούμε να γεννάμε δεδομένα οποιασδήποτε κατανομής (ομοιόμορφης, υψηλής-χαμηλής συσχέτισης κλπ) καθώς και να ορίσουμε την ταχύτητα/τυχαioτητα με την οποία φτάνουν τα ερωτήματα. Για τον σκοπό αυτό, στο δικό μας σύστημα έχουμε επιλέξει την τυχαία αποστολή ερωτημάτων (*flag* = 1) σε χρόνο μεταξύ 3 - 5 δευτερολέπτων. Αυτό γίνεται αφενός για λόγους ρεαλισμού, καθώς μια τέτοια συχνότητα αποτελεί λογική για άφιξη σε ένα πραγματικό Real time σύστημα και αφετέρου για να προλαβαίνει ο χρήστης/προγραμματιστής να ελέγχει τα metrics που τυπώνονται κατά την άφιξη κάθε query στην κονσόλα.

Ως κύρια κλάση εκτέλεσης του συστήματός μας έχουμε ορίσει την *SkylineMain()* εντός της οποίας ορίζεται ολόκληρη η σύνδεση του συστήματος με τα (input-output) Kafka topics, των components του μεταξύ τους καθώς και η υλοποίηση της αρχιτεκτονικής του Flink.

Αναλυτικότερα, σε πρώτη φάση αναχτούμε το (streaming) περιβάλλον εκτέλεσης και θέτουμε τον επιθυμητό παραλληλισμό της εφαρμογής μας (παράμετρος). Εν συνεχεία πραγματοποιούμε τις απαραίτητες συνδέσεις μεταξύ Flink και των input Kafka topics και ανάλογα με τον αλγόριθμο που έχουμε επιλέξει (παράμετρος) ακολουθεί το partitioning/Map κομμάτι της εφαρμογής. Από εκεί, η αρχική ροή δεδομένων μας διαμοιράζεται σε υπο-ροές, ανάλογα με το partition που ανήκει κάθε σημείο της ροής. Πάνω σε κάθε (υπό-)ροή υπολογίζεται το Local Skyline χρήσι του αλγορίθμου BNL. Πλέον κάθε (υπό-)ροή έχει το τοπικό της Skyline και έτσι απλώς συνδέοντας κατάλληλα την ροή ερωτημάτων με αυτή των skylines μπορούμε να σηματοδοτήσουμε τον υπολογισμό του συνολικού (global) skyline, σε κάθε άφιξη ερωτήματος. Το ποια Local Skyline χαρακτηρίζονται ως Global γίνεται και πάλι μέσω του BNL αλγορίθμου. Προφανώς, το τελικό skyline καταλήγει σε ένα output Kafka topic και εκεί η λογική της εφαρμογής μας σταματάει και αρχίζει η εκτέλεση.

Ο προγραμματισμός του συστήματος έγινε σε Java 17, στην έκδοση 3.0.0-1.17 του Flink, την 3.1.1 του Maven. Η εκτέλεσή του κώδικα και των επακόλουθων πειραμάτων έγινε λοσβάλλψ στο IntelliJ IDEA 2025.3.

Πειραματικά Αποτελέσματα

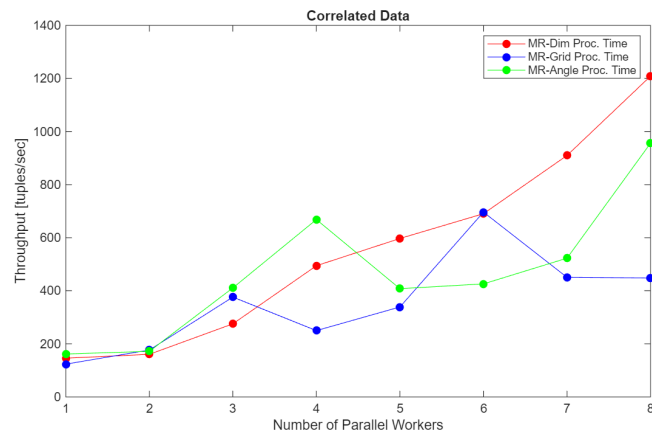
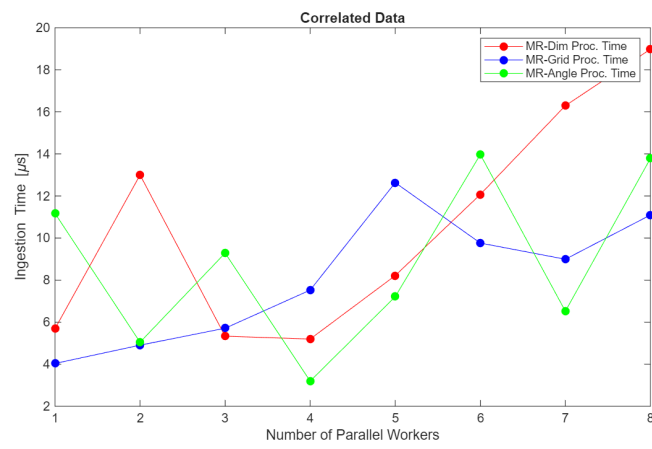
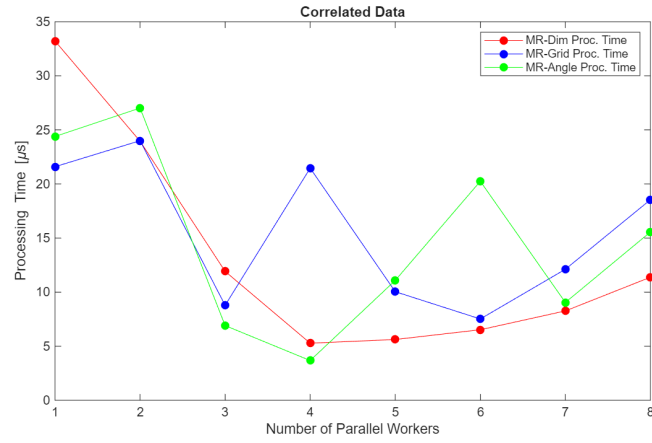
Δεδομένου ότι η δημοσίευση ασχολείται με πειράματα πάνω σε Σύνολα Δεδομένων (Batch Processing) οι μετρητικές συμφωνά με τις οποίες οι ερευνητές κρίνουν την αποδοτικότητα του συστήματός τους, όπως ο συνολικός χρόνος επεξεργασίας δεν μπορούν να εφαρμοστούν εξ ορισμού σε ένα streaming σύστημα σαν το δικό μας.

Επομένως, για την καταμέτρηση των δυνατοτήτων του pipeline μας, επιλέξαμε την αποδοτικότητα των αλγορίθμων μέσω των εξής μετρητικών, σε 'διάρκεια' (δηλαδή μετά την άφιξη) 30 ερωτημάτων:

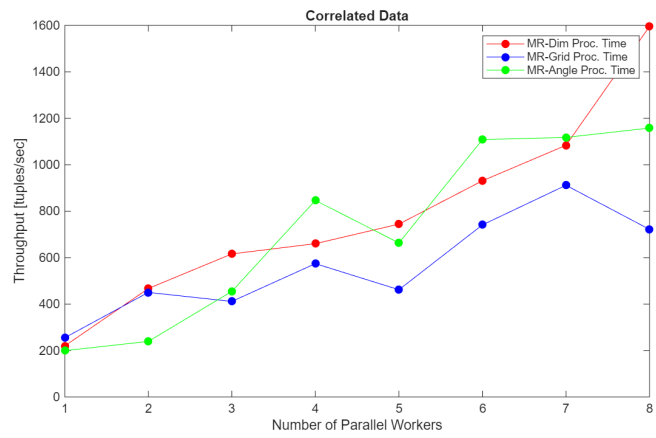
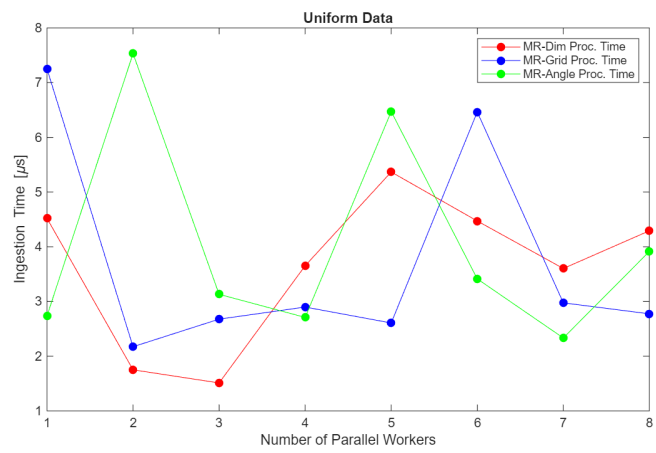
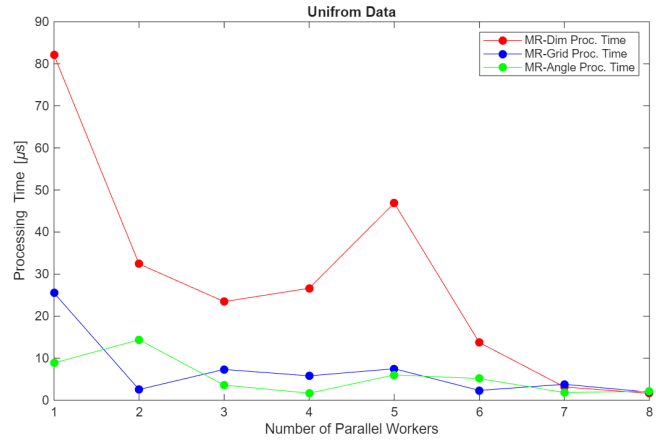
1. Μέσος χρόνος πεξεργασίας (**Map time** + **Reduce time**)
2. Μέσος Ingestion χρόνος (μετά από πόσο χρόνο που το input topic 'προσφέρει' ένα tuple προς επεξεργασία, το σύστημα ξεκινάει την επεξεργασία του)
3. Τηρουγητ συστήματος

Οι μετρήσεις που λάβαμε αφορούν σε πειράματα πάνω στους 3 αλγορίθμους με 3 διαφορετικές κατανομές παραγωγής δεδομένων (correlated, uniform και anti-correlated) και για διαφορετικές τιμές παραλληλισμού (αριθμό 'worker' κόμβων).

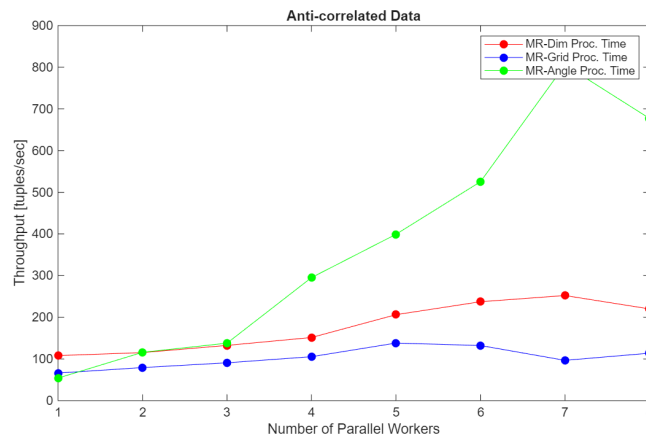
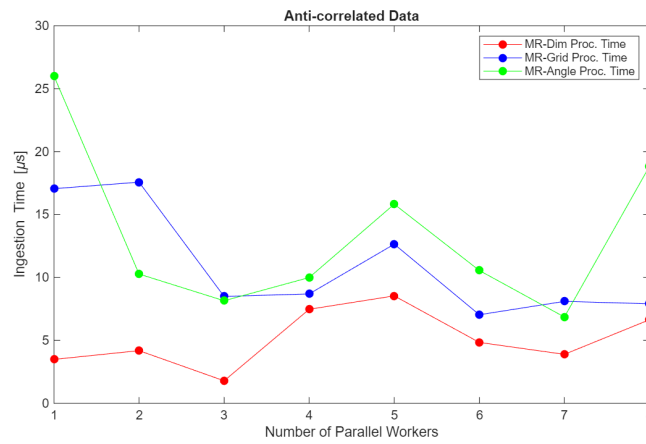
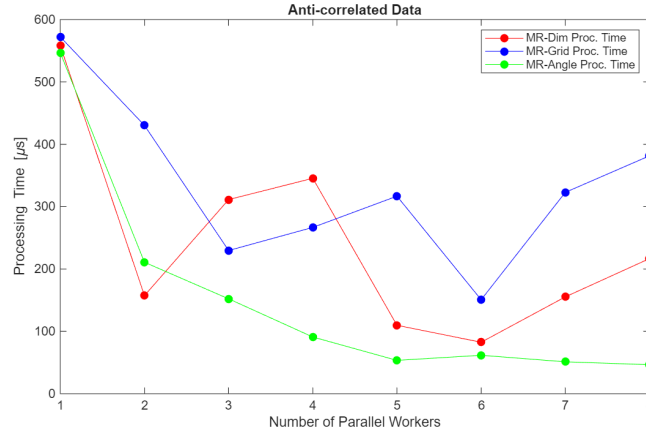
Correlated Data



Uniform Data



Anti-Correlated Data



Σχολιασμός Αποτελεσμάτων

Γενικά, τα αποτελέσματά μας είναι εμφανές ότι καταβάλλονται έντονα από θόρυβο. Αυτό είναι λογικό και αναμενόμενο όταν μελετάμε ένα κατανεμημένο και σχετικά ταχύ σύστημα επεξεργασίας, που οι υπολογισμοί γίνονται στην κλίμακα των 10^{-6} sec. Μελετώντας ένα τέτοιο σύστημα πρέπει να λαμβάνουμε υπόψιν φαινόμενα cache missing, task scheduling θορύβου κλπ.

Αναφορικά με τις γραφικές χρόνου (τόσο χρόνου επεξεργασίας όσο και ingestion χρόνου) παρατηρούμε ότι παρόλο που η χρήση παραλληλισμού ($p > 1$) είναι σαφώς καλύτερη από τη μη χρήση, για κάθε είδος κατανομής, μεταξύ κατανομών αυτή που οφελείται περισσότερο από όλες από τη χρήση παραλληλισμού είναι η ομοιόμορφη. Αυτ'οσοιμβάνει διότι, η ομοιόμορφη κατανομή διαμοιράζει τα σημεία (περίπου) ίσα μέσο στο χώρο με αποτέλεσμα, να παράσχει χωρίς επιπλέον διαδικασίες το όφελος του load balancing. Κατά αυτόν τον τρόπο η δουλειά κατανέμεται

περίπου ίσα μεταξύ των parallel worker nodes και έτσι ο ‘πιο αργός’ εξ αυτών δεν απαίχει πολύ από τη μέση περίπτωση, αποφεύγοντας τη δημιουργία μεγάλων bottlenecks κατά το Reduce.

Τα anti-correlated data είναι γενικά ευαίσθητα σε θόρυβο και οι αλγόριθμοι δεν μπορούν να αξιοποιήσουν πλήρως τις δυνατότητες της παραλληλοποίησης, όταν ακολουθείται αυτή η κατανομή, λόγω κακού διαμοιρασμού του φορτίου. Αυτό οδηγεί σε υψηλούς χρόνους επεξεργασίας και ingestion. Αναλυτικότερα, το ingestion time παραμένει υψηλό, καθώς αντικατοπτρίζει τον ‘κορεσμό’ του συστήματος, που σε μια anti-correlated κατανομή δεδομένων είναι σχεδόν αναπόφευκτος: αν το σύστημα είναι απασχολημένο να πραγματοποιεί υπολογισμούς σε (κακώς διαμοιραμένα) δεδομένα, τότε τα εισερχόμενα datapoints θα αργήσουν να εισέλθουν σε κάποιον operator.

Σε ότι έχει να κάνει με το throughput του συστήματος, βλέπουμε ότι τα anti-correlated δεδομένα δίνουν το χαμηλότερο throughput για όλα τα επίπεδα παραλληλισμού και αυτό διότι η συγκεκριμένη κατανομή έχει υψηλό Reduce χρόνο ελαττώνοντας έτσι τον αριθμό πλειάδων προς επεξεργασία.

Αντίθετα, η ομοιόμορφη κατανομή δεδομένων αξιοποιεί την αύξηση του παραλληλισμού πιο αποτελεσματικά, καθώς όταν αυτός αυξάνεται, ανεβαίνει και το throughput του συστήματος.

Ακόμη, μπορούμε να πούμε ότι η κατανομή των δεδομένων είναι αυτή που καθορίζει την απόδοση των αλγορίθμων πέρα από κάθε άλλο παράγοντα. Τα anti-correlated data δίνουν σαφώς υψηλότερους χρόνους επεξεργασίας, τα correlated τους πιο μικρούς ενώ τα ομοιόμορφα, ενδιάμεσους, λόγω των μεγεθών των skyline που αυτά δημιουργούν.

Ενδεικτικά αναφέρουμε ότι συγκρίνοντας το μέγεθος των skyline που δίνει κάθε αλγόριθμος (εκτυπώνονται στο τερματικό κατά την εκτέλεση) διαφέρει από κατανομή σε κατανομή. Αναλυτικότερα

$$SL_{anti-cor} > SL_{uniform} > SL_{cor}$$

Αυτό συμβαίνει γιατί στα anti-correlated data έχουμε το φαινόμενο του ότι, αν ένα datapoint είναι ‘καλό’ σε μία διάσταση, τότε θα είναι κακό στην άλλη, δίνοντας έτσι, πολλά σημεία που κυριαρχούν έναντι άλλων έστω και σε μία διάσταση (‘όλοι είναι καλοί έστω σε κάτι’). Το αντίθετο συμβαίνει στα correlated data, για τα οποία ισχύει ότι datapoints που είναι καλά σε μία διάσταση θα είναι καλά σε όλες, ξεχωρίζοντας γρήγορα τα κυρίαρχα σημεία από τα μη. Προφανώς, η ενδιάμεση κατάσταση των ομοιόμορφα κατανεμημένων σημείων δίνει μερικές σχέσεις κυριαρχίας από κάποια σημεία, ενώ άλλα δεν συγκρίνονται καν, δίνοντας έτσι ένα μετριασμένο μέγεθος skyline.

Συγκρίνοντας τους 3 αλγόριθμους δεν μπορούμε να εντοπίσουμε κάποιον ολοκληρωτικά καλύτερο, σε όλα τα σενάρια. Ο MR-Dim δείχνει μια γενική σταθερότητα και αντοχή σε διαφορετικές κατανομές, εκμεταλλευόμενος με σε επαρκή βαθμό τον παραλληλισμό, με συνέπεια ανά τα τεστ. Ο MR-Grid πιάνει καλή απόδοση σε ορισμένες περιπτώσεις αλλά υπόκειται σε σφάλματα λόγω ευαισθησίας στο πως έχουν γίνει partition τα δεδομένα στο χώρο. Ο MR-Angle αποδίδει καλά για ομοιόμορφη και correlated κατανομή δεδομένων, αλλά δυσκολεύεται στην anti-correlated, παρουσιάζοντας μεγάλη ευαισθησία σε skewed δεδομένα.

Επίλογος

Συνοπτικά, μπορούμε να πούμε ότι τα πειραματικά αποτελέσματα αν και όχι οπτικά πειστικά, επιβεβαιώνουν την διαίσθησή μας για το πως ένα τέτοιο σύστημα συμπεριφέρεται και αποδίδει πάνω σε διαφορετικές κατανομές και partitioning αλγορίθμους. Μεταξύ των αλγορίθμων δεν υπάρχει ξεκάθαρος ‘επικρατών’ όπως στο (batch) σύστημα της δημοσίευσης που ο MR-Angle ήταν ξεκάθαρα ο πιο αποδοτικός. Αυτό τονίζει για τα καλά τη διαφορετικότητα μεταξύ ‘στατικών’ δεδομένων και δεδομένων σε ροές, και πως ο σχεδιαστής πρέπει να είναι σε θέση να κατανοήσει την καταλληλότητα (ή μη) κάθε αλγορίθμου στο εκάστοτε περιβάλλον.